Frogger Lab — Chat History

USER

I am working on a Pygame Frogger lab.

Please inspect the entire codebase I provided, especially game/collisions.py, game/game_engine.py, game/frog.py, and game/vehicle.py. There is one deliberate bug: vehicle collision detection compares grid columns instead of checking whether the frog and vehicle actually overlap.

Fix ONLY the vehicle collision detection. Use actual x/y positions and dimensions, detect only actual visible rectangle overlap, support different vehicle speeds, add no lives/score/timer, preserve architecture. First explain the bug, then make the smallest changes, then give manual tests.

ASSISTANT

The bug is in game/collisions.py: it converts the vehicle's continuous x position into a grid column and compares that with the frog's grid column. That cannot determine actual visual overlap because vehicles move continuously and are not grid-snapped.

The smallest fix is to use the existing visible rectangles:
frog_rect = frog.get_rect(CELL_SIZE)
for v in vehicles:
vehicle_rect = v.get_rect(CELL_SIZE)
if frog_rect.colliderect(vehicle_rect):
return True

No changes are needed in game_engine.py, frog.py, or vehicle.py. Manual tests should cover separated objects in the same lane, clear and partial overlap, same approximate grid column without overlap, different vehicle speeds/directions, and edge-touching.

USER

Now implement ONLY Task 2 from the Frogger README: a 3-life system with respawn.

Requirements: start with exactly 3 lives; collision decreases a life; reset frog after collision; continue if lives remain; display lives; after all lives display Game Over; make the collision visually understandable before respawn; R restarts with 3 lives; no score or timer.

Inspect the architecture, reuse Task 1 collision detection, make minimal changes. Explain changed files, life state, collision → visible hit → respawn, and manual tests.

ASSISTANT

Task 2 was integrated around the existing GameEngine. Only game/game_engine.py needed to change; Task 1 collision detection remains the collision source.

The engine tracks lives starting at 3 and a short hit timer. On collision, one life is removed and a visible HIT state is shown before the frog is reset. If lives reach zero, Game Over is displayed. R resets lives, hit state, and entities.

Sequence: vehicle overlaps frog → life decreases → HIT state → short delay → frog respawns → next attempt if lives remain. Tests cover initial 3 lives, each collision, Game Over, R restart, rectangle collision regression, and absence of score/timer.

USER

Now implement ONLY Task 3 from the Frogger README: goal detection and score tracking.

Requirements: detect the goal at the top; register crossing exactly once; increase and display score; end the game; display You Won!; do not add/modify timer; preserve 3-life collision/respawn; R resets score. Inspect and integrate with current game-state architecture. Explain changes and give a short manual checklist.

ASSISTANT

Task 3 was integrated into the existing Task 2 architecture. The engine tracks score and a persistent won state. When frog.row reaches GOAL_ROW, score increases once, won becomes true, and gameplay stops. This prevents repeated scoring on later frames.

The score is displayed during gameplay. You Won! is displayed after reaching the goal. R resets lives, score, win state, hit state, and entities. Collision/life behavior remains intact and no timer was added.

Tests: start with Lives 3 / Score 0; reach goal and verify Score 1 / You Won!; wait and verify score stays 1; test collision and Game Over; press R and verify Lives 3 / Score 0; verify no timer.

USER

Now implement ONLY Task 4 from the Frogger README: a 30-second countdown timer.

Requirements: each attempt starts at 30 seconds; display remaining time; continuous countdown; timeout is a genuine failed attempt; timeout consumes a life; if lives remain start the next attempt at the starting position; if all lives are used display Game Over; reaching goal before zero wins regardless of remaining time; R resets timer; do not break collision, respawning, lives, score, or controls. Make minimal architectural changes. Explain timer calculation, timeout/lives, winning, restart, tests, and compile the whole code instead of patches.

ASSISTANT

Task 4 was integrated into the existing Task 3 architecture. The timer uses real elapsed time via pygame.time.get_ticks(), rather than frame counting, so it remains a true 30-second attempt despite FPS variation.

On timeout, one life is consumed. If lives remain, the frog returns to its starting position and a fresh 30-second attempt begins. If lives reach zero, Game Over is displayed.

Goal detection takes priority over timeout: reaching the goal before time expires increases score once, displays You Won!, and stops gameplay and the timer.

R resets lives to 3, score to 0, win/game-over state, frog and vehicles, and the attempt start timestamp.

The complete project was compiled with `python -m compileall -q .` and reported `COMPILE_OK`. A complete project ZIP was generated as frogger_task4_complete.zip.

Manual tests cover initial 30 seconds, continuous countdown, timeout consuming lives, final Game Over, collision/HIT behavior, winning before timeout, timer stopping after win, R resetting everything, and regression checks for controls, vehicles, collisions, lives, score, and win/game-over behavior.